Week 3 - AWS CDK (constructs, cdk synth / diff / deploy, CDK Pipelines)

DOP-C02 Domain 2 (Configuration Management and IaC). You must:

1. **Know when to choose CDK**: infrastructure in a **programming language** with loops, classes, reuse and **unit tests**.
2. **Know construct levels** (L1 / L2 / L3), escape hatches and core concepts (logical IDs, environments, context, tokens).
3. **Use the CLI**: `cdk bootstrap`, `cdk synth`, `cdk diff`, `cdk deploy` and their options.
4. **Build CDK Pipelines**: self-mutating, multi-account pipelines.

The big idea: **CDK = code -> CloudFormation.** `cdk synth` produces CloudFormation templates and **CloudFormation does the deploying**. Everything from the CloudFormation topics still applies (change sets, rollback, drift, limits).

Part 1 - What CDK is

- Open-source framework: **TypeScript, JavaScript, Python, Java, C# / .NET, Go**.
- **CDK v2**: one library `aws-cdk-lib` plus the `constructs` package (v1 is end-of-support).
- **CDK CLI** (`npm install -g aws-cdk`) runs your app, synthesizes templates, publishes assets and deploys through CloudFormation.
- Output: a **cloud assembly** in `cdk.out/` (templates, asset manifests, tree metadata).

```
App
  Stage "Prod" (optional: stacks for one environment)
    Stack "NetworkStack"  -> one CloudFormation stack
      Vpc (L2) -> CfnVPC, CfnSubnet ... (L1)
    Stack "AppStack"
      Bucket (L2)
      Function (L2) -> CfnFunction + IAM role
```

Element	Meaning
App	Root; running the app (`cdk.json` "app" command) builds the construct tree
Stage	Group of stacks deployed together to one **environment** (account + Region); used by CDK Pipelines
Stack	**= one CloudFormation stack**, the unit of deployment
Construct	Building block; every construct takes `(scope, id, props)`

```
const bucket = new s3.Bucket(this, 'DataBucket', { versioned: true });  // scope, id, props
const fn = new lambda.Function(this, 'Processor', { /* ... */ });
bucket.grantRead(fn);   // least-privilege IAM policy on the function role
```

Part 2 - Construct levels and escape hatches

Level	What it is	Example	When
L1 (CFN resources)	**One-to-one with CloudFormation**, auto-generated, `Cfn` **prefix**, **no defaults**	`s3.CfnBucket`, `ec2.CfnVPC`	New property / service with no L2, full control
L2 (curated)	Intent-based API, **secure defaults**, **helper methods** (`grant*`, `add*`, `metric*`)	`s3.Bucket`, `lambda.Function`, `ec2.Vpc`	**Default choice**
L3 (patterns)	**Several resources** wired as a common architecture	`ApplicationLoadBalancedFargateService`, `LambdaRestApi`	Whole pattern in a few lines

- L2 helpers: `bucket.grantReadWrite(role)`, `table.grantReadData(fn)` (IAM without hand-written JSON), `fn.addEventSource(...)`, `queue.metricApproximateNumberOfMessagesVisible()`.
- `ec2.Vpc` default: **public + private subnets in every AZ with NAT gateways** (cost trap; set `natGateways`).
- **Your own constructs**: extend `Construct` to package a company standard (for example a SecureBucket), publish to **CodeArtifact** for reuse. **Construct Hub** is the public catalog.

Escape hatches (L2 missing a property)

```
const cfnBucket = bucket.node.defaultChild as s3.CfnBucket;   // the L1 under the L2
cfnBucket.analyticsConfigurations = [ /* ... */ ];           // set an L1 property
cfnBucket.addPropertyOverride('NewProperty.Enabled', true);   // raw override (not yet in L1)
cfnBucket.addOverride('DeletionPolicy', 'Retain');            // any part of the resource JSON
```

Order: **L2 prop -> L1 via** `node.defaultChild` **->** `addPropertyOverride` **/** `addOverride`.

Referencing vs importing existing resources

- `Bucket.fromBucketName`, `Vpc.fromLookup`, `Role.fromRoleArn` give a **reference** for grants and wiring; **CDK doesn't manage** them.
- To bring an existing resource **under CDK management**: `cdk import` (CloudFormation resource import).
- To reuse an existing CloudFormation template inside CDK: `CfnInclude`. To generate a CDK app from a template or stack: `cdk migrate`.

Part 3 - Core concepts

Logical IDs (replacement trap)

- Logical ID = **construct path + hash** (for example `DataBucketE3889A50`).
- **Renaming a construct id or moving it to another scope changes the logical ID**: CloudFormation sees a new resource and **deletes the old one** (data loss for databases and buckets).
- Fix: keep ids stable, `overrideLogicalId('OldId')` when refactoring, and always run `cdk diff` (look for destroy + add of the same resource).

RemovalPolicy

- `RETAIN` is the default for **stateful resources** (S3, DynamoDB...). Maps to DeletionPolicy and UpdateReplacePolicy.
- `DESTROY` + `autoDeleteObjects: true` for S3 (a **custom resource** empties the bucket) - dev / test only.
- `SNAPSHOT` for RDS, EBS.
- "Bucket still exists after `cdk destroy`" = default **RETAIN**.

Environments (account + Region)

```
new AppStack(app, 'AppStack', { env: { account: '111111111111', region: 'us-east-1' } });
// or from the CLI profile
env: { account: process.env.CDK_DEFAULT_ACCOUNT, region: process.env.CDK_DEFAULT_REGION }
```

- **Environment-agnostic** stack (no `env`) deploys anywhere but **can't do lookups** (`Vpc.fromLookup`, `HostedZone.fromLookup`) and only uses 2 AZs.
- **Lookups need an explicit `env`.**

Context and cdk.context.json

- Lookups (VPC, AMI, hosted zone, SSM `valueFromLookup`) run at **synth time** and are **cached in** `cdk.context.json`.
- **Commit** `cdk.context.json` so synth is **deterministic** (pipeline = laptop). Refresh with `cdk context --reset` / `--clear`.
- Context values: `cdk.json` "context", `-c key=value`, read with `this.node.tryGetContext('env')`.
- **Feature flags** in `cdk.json` change default behaviour per project.

Tokens, parameters and secrets

- Deploy-time values (ARNs, Ref, GetAtt) are **tokens** (placeholders at synth). You can't use `if` on them in code; use `CfnCondition` / `Fn.conditionIf`.
- `StringParameter.valueForStringParameter()` = **deploy time** (CFN parameter); `valueFromLookup()` = **synth time** (context).
- Secrets: `SecretValue.secretsManager('name')` -> CloudFormation **dynamic reference**. **Never hardcode secrets.**
- `CfnParameter` works but is discouraged; CDK prefers **synth-time values** (context, props, env vars).

Cross-stack references

- Passing an object (vpc, bucket) from StackA to StackB creates **Exports + Fn::ImportValue automatically**.
- **Deadly embrace**: removing the usage fails because the export is still imported. Keep it with `this.exportValue(...)`, deploy, then remove (two steps).
- **Cross-Region** references: `crossRegionReferences: true` (SSM parameters + custom resources).
- `NestedStack` construct for the 500-resource limit.

Aspects, Tags, permissions boundaries

- **Aspects** visit **every construct in a scope** to **check or modify** it: `Aspects.of(app).add(new Checker())`. `Annotations.of(node).addError()` **fails synth**.
- **cdk-nag**: aspect-based rule packs (AWS Solutions, HIPAA, NIST) - **shift-left compliance**.
- `Tags.of(app).add('CostCenter', '123')` tags every taggable resource below.
- `PermissionsBoundary.of(scope).apply(policy)` or the bootstrap option.

Assets

- **File assets** (Lambda code, s3-deployment) and **Docker image assets** are built locally or in the pipeline, then **uploaded to the bootstrap S3 bucket and ECR repository** during deploy.
- **Docker is required** where image assets are built.

Part 4 - Bootstrapping

```
cdk bootstrap aws://111111111111/us-east-1
```

- Creates the `CDKToolkit` stack, **one per account + Region**:
- **S3 staging bucket** (file assets, large templates)
- **ECR repository** (image assets)
- **IAM roles**: deploy role, file-publishing role, image-publishing role, lookup role, **CloudFormation execution role**
- SSM parameter with the **bootstrap version**
- **Every account and Region you deploy to must be bootstrapped.** Errors like "SSM parameter /cdk-bootstrap/hnb659fds/version not found" = **not bootstrapped** (or wrong qualifier).
- **Re-bootstrap** when a newer CDK version needs a newer bootstrap version.

Cross-account trust (pipelines)

```
# In each TARGET account: trust the pipeline / tooling account
cdk bootstrap aws://222222222222/us-east-1 \
  --trust 999999999999 \
  --trust-for-lookup 999999999999 \
  --cloudformation-execution-policies arn:aws:iam::aws:policy/AdministratorAccess
```

Option	Meaning
`--trust ACCOUNT`	That account can **assume the deploy and publishing roles** (pipeline can deploy here)
`--trust-for-lookup`	Lookup role only (read-only)
`--cloudformation-execution-policies`	Policies on the **CFN execution role** - least privilege: scoped policy instead of AdministratorAccess
`--qualifier`	Several bootstrap stacks side by side (per team)
`--custom-permissions-boundary`	Boundary on IAM roles created by the app
Custom template	`cdk bootstrap --show-template`, then customise (KMS key, bucket policy)

Part 5 - CLI commands

Command	What it does	Exam use
`cdk init app --language typescript`	Project skeleton	
`cdk ls`	List stacks	
`cdk synth`	**Runs the app**, writes **templates + asset manifests to** `cdk.out`; no AWS changes	Review / validate (cfn-lint, cfn-guard, checkov); **pipeline build step**
`cdk diff`	Compares synthesized template with the **deployed stack**: adds, deletes, **replacements**, **IAM / SG changes**	Catch replacements before deploy; PR review; `--fail` exits non-zero on differences
`cdk deploy`	Synth -> **publish assets** -> **create + execute change set** -> wait	Deploy
`cdk destroy`	Delete stacks (RETAIN resources stay)	
`cdk bootstrap`	Prepare an environment	
`cdk watch` / `cdk deploy --hotswap`	Updates Lambda code, ECS images, Step Functions **directly, bypassing CloudFormation**	**Dev only** (drift)
`cdk import`	Bring existing resources into a stack	
`cdk migrate`	Generate a CDK app from a CFN template / stack / resources	Convert CFN to CDK
`cdk context`	Show / clear cached context	
`cdk doctor`	Environment diagnostics	
`cdk drift` (newer)	Drift detection on deployed stacks	
`cdk gc` (newer)	Remove unused assets from the bootstrap bucket / ECR	

Key cdk deploy options

Option	Meaning
`--all` / stack names	Which stacks (dependency order is automatic)
`--require-approval`	`never` / `broadening` (default, prompts when **IAM or SG permissions are broadened**) / `any-change`
`--no-execute`	Create the **change set without executing**
`--method=direct`	Skip change sets
`--parameters Stack:Key=Value`	Values for CfnParameters
`--outputs-file out.json`	Stack outputs to a file (for tests)
`--no-rollback`	Keep failed resources to debug
`--exclusively`	Only the named stack, not its dependencies
`--concurrency N`	Deploy independent stacks in parallel
`--role-arn`, `--tags`, `--notification-arns`	CloudFormation options

Stack props: `terminationProtection: true`, `stackName`, `description`, `synthesizer` (`DefaultStackSynthesizer` uses the bootstrap roles; `CliCredentialsStackSynthesizer` uses your credentials).

Part 6 - Testing CDK code

Test type	How
Fine-grained assertions	`aws-cdk-lib/assertions`: `Template.fromStack(stack)`, `hasResourceProperties`, `resourceCountIs`, `hasOutput`, `Match.objectLike`
Snapshot tests	Compare the whole template with a stored snapshot (Jest `toMatchSnapshot`) - catches unexpected changes
Compliance	**cdk-nag** aspects; cfn-lint / cfn-guard / checkov on `cdk.out`
Integration tests	`integ-tests` (`IntegTest`) or post-deploy steps in the pipeline

Run unit tests in the **pipeline synth / build step** before deploying.

Part 7 - CDK Pipelines (aws-cdk-lib/pipelines)

A high-level construct that builds a **CodePipeline** to deploy CDK apps across accounts and Regions, and **updates itself**.

```
const pipeline = new CodePipeline(this, 'Pipeline', {
  pipelineName: 'MyAppPipeline',
  crossAccountKeys: true,                 // KMS-encrypted artifacts, needed cross-account
  synth: new ShellStep('Synth', {
    input: CodePipelineSource.connection('my-org/my-app', 'main', {
      connectionArn: 'arn:aws:codeconnections:us-east-1:999999999999:connection/abc',
    }),
    commands: ['npm ci', 'npm run build', 'npm test', 'npx cdk synth'],
  }),
});

const dev  = { account: '222222222222', region: 'us-east-1' };
const prod = { account: '333333333333', region: 'us-east-1' };

pipeline.addStage(new MyAppStage(this, 'Dev', { env: dev }));

pipeline.addStage(new MyAppStage(this, 'Prod', { env: prod }), {
  pre:  [new ManualApprovalStep('PromoteToProd')],
  post: [new ShellStep('SmokeTest', {
    commands: ['curl -f https://api.example.com/health'] })],
});

class MyAppStage extends Stage {          // the stacks for one environment
  constructor(scope: Construct, id: string, props?: StageProps) {
    super(scope, id, props);
    new NetworkStack(this, 'Network');
    new AppStack(this, 'App');
  }
}
```

```
Source -> Build (npm test + cdk synth) -> UpdatePipeline (self-mutation)
       -> Assets (publish files / images) -> Dev -> Prod (approval, deploy, post tests)
```

Feature	Detail
Self-mutation	**UpdatePipeline** stage deploys the **pipeline stack itself**; pipeline code changes update the pipeline and restart it (`selfMutation: true` default)
First deployment	`cdk deploy PipelineStack` **once manually**; after that **git push** drives everything
Sources	`CodePipelineSource.connection()` (**CodeConnections**: GitHub, GitLab, Bitbucket), `.codeCommit()`, `.s3()`, `.ecr()`, `.gitHub()` (OAuth token, legacy)
Synth step	`ShellStep` or `CodeBuildStep` (custom role, VPC, image, env vars); output `cdk.out`; `dockerEnabledForSynth` if Docker needed
Stages / waves	`addStage()` deploys stacks in dependency order; `addWave()` deploys several stages **in parallel** (Regions)
pre / post steps	`ManualApprovalStep`, `ShellStep` / `CodeBuildStep` for integration, smoke and security tests; `stackSteps` per stack
Outputs in tests	`envFromCfnOutputs` passes stack outputs into post-step env vars
Cross-account	Targets **bootstrapped with** `--trust <pipeline account>` + `crossAccountKeys: true`
Customise	`buildPipeline()` then `pipeline.pipeline`; `useChangeSets` default true

Need	Answer
CDK app, multi-account, pipeline in the same code, updates itself	**CDK Pipelines**
Org-wide baseline in every account	**StackSets**
Serverless only, local testing	**SAM** (`sam local` can use `cdk synth` output)
Users self-service products	**Service Catalog** (CDK `ProductStack`)

Part 8 - CDK vs CloudFormation vs SAM vs Terraform

	CloudFormation	SAM	CDK	Terraform
Language	YAML / JSON	YAML (transform)	**TS, Python, Java, C#, Go**	HCL
Abstractions	Low	Serverless shorthand	**L2 / L3, classes, loops**	Modules
Deploys via	Itself	CloudFormation	**CloudFormation**	Terraform state
Unit tests	cfn-lint / guard	Same	**Assertions in the language**	terraform test
Keyword	Native IaC	Serverless, sam local	**Familiar language, reusable constructs, test infra like code**	Multi-cloud

Part 9 - Exam traps

Trap	Truth
CDK deploys resources with direct API calls	**CDK synthesizes CloudFormation**; CloudFormation deploys (except hotswap)
Deploy fails: bootstrap version not found	`cdk bootstrap` that account + Region (or re-bootstrap)
Pipeline can't deploy into prod account	Prod **bootstrapped with** `--trust <pipeline account>` + `crossAccountKeys: true`
Renamed construct id, table replaced	**Logical ID changed**; keep ids / `overrideLogicalId`; `cdk diff` first
Bucket remains after `cdk destroy`	Default **RemovalPolicy RETAIN**
`Vpc.fromLookup` fails at synth	Stack is environment-agnostic: **set** `env`
Pipeline synth differs from local	**Commit** `cdk.context.json`
L2 lacks a property	**Escape hatch**: `node.defaultChild` / `addPropertyOverride`
L1 is high level	**L1 = Cfn* raw**; L2 curated; L3 patterns
`cdk synth` changes AWS	**No**, local templates (lookups only read)
See replacements / IAM changes before deploy	`cdk diff`
CI deploy hangs on approval	`--require-approval never` (default `broadening`)
`--hotswap` / `cdk watch` in prod	**Dev only**: bypasses CloudFormation, drift
Redeploy pipeline after pipeline code change	**Self-mutation** does it; only the first deploy is manual
Stages to several Regions in parallel	`addWave()`
Approval before prod	`ManualApprovalStep` in `pre`
Hardcoded secret	`SecretValue.secretsManager` / SSM dynamic reference
Enforce encryption / tags everywhere	**Aspects** (cdk-nag) / `Tags.of`
Removing a cross-stack reference fails	Deadly embrace: `exportValue`, two-step deploy
Cross-Region stack reference	`crossRegionReferences: true`
Convert CFN template to CDK	`cdk migrate` (or `CfnInclude`)
Manage an existing resource with CDK	`cdk import` (`fromXxx` only references)

Part 10 - Hands-on lab (about 60 minutes, very low cost)

Lab 1 - First app, synth

1. `npm install -g aws-cdk`, then `mkdir cdk-lab && cd cdk-lab && cdk init app --language typescript`.
2. In `lib/cdk-lab-stack.ts` add:

```
const bucket = new s3.Bucket(this, 'DataBucket', {
  versioned: true, encryption: s3.BucketEncryption.S3_MANAGED,
  removalPolicy: RemovalPolicy.DESTROY, autoDeleteObjects: true });
const fn = new lambda.Function(this, 'Reader', {
  runtime: lambda.Runtime.PYTHON_3_12, handler: 'index.handler',
  code: lambda.Code.fromInline('def handler(e,c):\n  return "ok"') });
bucket.grantRead(fn);
new CfnOutput(this, 'BucketName', { value: bucket.bucketName });
```

3. `cdk synth`; open `cdk.out/CdkLabStack.template.json`. Find the **hashed logical IDs**, the **grantRead IAM policy** and the **autoDeleteObjects custom resource**.

Lab 2 - Bootstrap, deploy, diff

1. `cdk bootstrap aws://<acct>/us-east-1`, then `cdk deploy`. Inspect the **CDKToolkit** stack (bucket, ECR repo, roles). Approve the **IAM changes** prompt (`broadening`).
2. Add `bucket.grantWrite(fn)` and run `cdk diff`: the IAM statement shows as broadening.

Lab 3 - Logical ID trap and escape hatch

1. Rename `'DataBucket'` to `'Data'`; `cdk diff` shows **destroy + add** (replacement).
2. Fix with `(bucket.node.defaultChild as s3.CfnBucket).overrideLogicalId('<old id>')`; `cdk diff` shows no replacement.
3. Escape hatch: `addPropertyOverride('ObjectLockEnabled', false)`, synth and check the JSON.

Lab 4 - Tests and an aspect

1. Assertion test: `Template.fromStack(stack).hasResourceProperties('AWS::S3::Bucket', { VersioningConfiguration: { Status: 'Enabled' } })`; run `npm test`.
2. Aspect: add an error annotation for any `CfnBucket` without versioning; remove `versioned` and watch `cdk synth` **fail**. Optionally try **cdk-nag** `AwsSolutionsChecks`.

Lab 5 - CDK Pipelines (single account is fine)

1. Push the project to GitHub and create a **CodeConnections** connection.
2. Add a `PipelineStack` (Part 7): one Stage with your account, ShellStep synth (`npm ci`, `npm test`, `npx cdk synth`), a `ManualApprovalStep` pre-step.
3. `cdk deploy PipelineStack` once, then push a commit. Watch **Source, Build, UpdatePipeline, Assets, Stage**.
4. **Self-mutation**: add a second stage in code and push; UpdatePipeline changes the pipeline and restarts.
5. Cleanup: `cdk destroy --all`, delete stage stacks, the connection, and optionally CDKToolkit (empty its bucket first).

Part 11 - Check yourself

Questions

1. What does `cdk synth` produce, where, and does it change AWS?
2. L1 vs L2 vs L3 constructs, with an example each?
3. An L2 construct lacks a new CloudFormation property. How do you set it?
4. What does `cdk bootstrap` create, and when is it required?
5. A pipeline in account 999 must deploy to prod account 333. Bootstrap command and pipeline setting?
6. What does `cdk diff` show, and why run it before every deploy?
7. After renaming a construct id, CloudFormation wants to replace a DynamoDB table. Why, and the fix?
8. Why does an S3 bucket remain after `cdk destroy`? How do dev buckets delete cleanly?
9. `Vpc.fromLookup` fails during synth. What's missing?
10. Why commit `cdk.context.json`?
11. What does `--require-approval broadening` do?
12. What is self-mutation in CDK Pipelines, and which deploy is manual?
13. How do you deploy a stage to three Regions in parallel and add an approval before prod?
14. How do you enforce "all buckets encrypted" across every construct at synth time?
15. When are `--hotswap` / `cdk watch` appropriate, and why not in prod?

Answers

1. **CloudFormation templates + asset manifests** (cloud assembly) in `cdk.out`; **no AWS changes** (lookups only read).
2. **L1** = `Cfn*` one-to-one, no defaults (`CfnBucket`); **L2** = curated with defaults and helpers (`Bucket`, `grantRead`); **L3** = multi-resource patterns (`ApplicationLoadBalancedFargateService`).
3. **Escape hatch**: `node.defaultChild` cast to L1 and set it, or `addPropertyOverride` / `addOverride`.
4. **CDKToolkit** stack: S3 bucket, ECR repo, deploy / publishing / lookup / CFN execution roles, version parameter; **once per account + Region** (and when a newer version is needed).
5. `cdk bootstrap aws://333/region --trust 999 --cloudformation-execution-policies <policy>` and `crossAccountKeys: true`.
6. Differences vs the deployed stack: adds, removals, **replacements**, **IAM / SG changes**; catches data loss and permission broadening.
7. **Logical ID comes from the construct path**; restore the id or `overrideLogicalId`.
8. Default **RemovalPolicy RETAIN**; for dev use `DESTROY` **+** `autoDeleteObjects: true`.
9. An explicit `env` **(account + Region)**.
10. It **caches lookups** so synth is **deterministic** (pipeline matches local).
11. Default; **prompts when IAM or security group permissions are broadened**.
12. **UpdatePipeline** stage redeploys the pipeline stack when pipeline code changes; only the **first** `cdk deploy PipelineStack` is manual.
13. `addWave()` with the three stages; `ManualApprovalStep` in the prod stage's `pre`.
14. An **Aspect** (cdk-nag or custom with an error annotation) - fails synth.
15. **Fast dev iteration only**; they bypass CloudFormation, causing **drift** and skipping change sets / rollback.

Quick memory summary

- **CDK** = infrastructure in TS / Python / Java / C# / Go -> **synthesizes CloudFormation** (`cdk.out`) -> deployed by CloudFormation.
- **App -> Stage -> Stack (= CFN stack) -> Construct (scope, id, props)**; **L1 Cfn***, **L2 curated**, **L3 patterns**; **escape hatches**.
- **Bootstrap** every account + Region (CDKToolkit); cross-account needs `--trust`.
- **synth** = templates, **diff** = what changes, **deploy** = assets + change set (`--require-approval broadening`); **hotswap is dev only**.
- Traps: **stable logical IDs**, **RETAIN default**, `env` **for lookups**, **commit cdk.context.json**, **Aspects / cdk-nag**, **SecretValue**.
- **CDK Pipelines**: Source -> Synth -> **UpdatePipeline (self-mutation)** -> Assets -> Stages / **Waves** with pre / post steps; first deploy manual; `--trust` + `crossAccountKeys`.